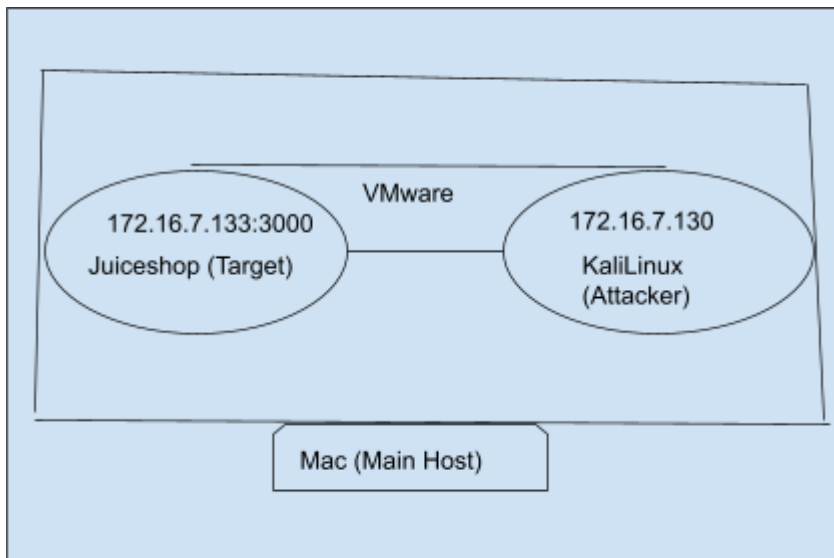


Web Application Penetration Test Final Report

Prepared for OWASP JuiceShop
By Mekonen Tegenie
September 16th 2026

Table of Contents	1
2.1 Network Topology.....	2
2.2 Target Details.....	2
2.3 Kali Linux Tester Details.....	2
2.4 Tools Used	2
3.1 Executive Summary	3
4.1. The scope of the assessment	4
4.2 methodology and standards	4
5.1 reconnaissance and enumeration	5
5.2. Basic Web FingerPrinting.....	6
5.3 Connectivity Validation.....	7
6.1. Findings and recommendations	8
6.2 overall risk ratings	8
6.2 overall risk ratings.....	9
6.3 Business Impact Summary.....	9
6.4 finding summary	10
7.1 CRITICAL RISK FINDINGS	11
7.2 sql injection Flaws.....	11 - 12
7.3 weak password configuration policy.....	13
8.1 HIGH RISK FINDINGS.....	14
8.2 Sensitive information Disclosure.....	14
9.1 MEDIUM RISK FINDINGS	15
9.2 vulnerability to DOM-based XSS.....	15
10.1 LOW RISK FINDINGS	16
10.2 Usernames Exposure.....	16
11.1 REMEDITION STRATEGIC GUIDANCE	17
12 Commands Used.....	18

2.1 Network Topology



2.2 Target Details

Target: OWASP Juice Shop
Target URL: <http://172.16.7.133:3000>
Target IP: 172.16.7.133
Network: Host-only lab network

2.3 KaliLinux Tester Details

Tester: Kali Linux
TEster IP : 172.16.7.130
Tester Subnet : 255.255.255.0

2.4 Tools used

Nmap
Burpsuite
Curl
Nikto
Whatweb

3.1.Executive Summary

A penetration test was performed against the OWASP Juice Shop application hosted at <http://172.16.7.133:3000> in an isolated host-only lab environment.

The following chart shows the count of findings by risk for this report.

Critical	High	Medium	Low
2	1	1	1

The assessment covered reconnaissance, endpoint enumeration, authentication, session management, authorization, input validation, injection, client-side security, business logic, security configuration.

A total of 5 findings were confirmed:

Critical: 2

High: 1

Medium: 1

Low: 1

4.1 The scope of the assessment

Target: OWASP Juice Shop on Docker container

Target URL: http://172.16.7.133:3000

Target IP: 172.16.7.133

Network: Host - only lab network

Tester: Kali Linux

Tester IP : 172.16.7.130

Authorized testing: Local lab environment only

Excluded targets: All systems other than 172.16.7.133:3000

4.2 Methodology and standards

Reconnaissance

Enumeration

Initial access

Privilege escalation

report

5.1 Reconnaissance and enumeration

After scanning the host with nmap i found out there are open ports that can be used to access the target as shown below

```
(johnny@PT-Kali01)-[~/pentest/juiceshop]
$ sudo nmap -Pn -sS -sV -p- --min-rate 1000 \
-oA recon/full-port-scan $TARGET

Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-09 04:42 -0400
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 172.16.7.133
Host is up (0.00068s latency).
Not shown: 65529 filtered tcp ports (no-response)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 10.2p1 Ubuntu 2ubuntu3.6 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http         nginx 1.28.3 (Ubuntu)
1514/tcp  closed fujitsu-dtcns
1515/tcp  closed ifor-protocol
3000/tcp  open  ppp?
55000/tcp closed unknown
```

5.2 Basic web fingerprinting

The response tells us the application is reachable and uses HTML5
And the URL is for sure leads us to OWASP Juice Shop

```
whatweb -a 3 http://172.16.7.131:3000
```

response

```
http://172.16.7.133:3000 [200 OK] Country[RESERVED][ZZ], HTML5, IP[172.16.7.131],  
Script[module], Title[OWASP Juice Shop],  
UncommonHeaders[access-control-allow-origin,x-content-type-options,feature-policy,x-recru  
ting], X-Frame-Options[SAMEORIGIN]
```

As we can see after testing the target with Nikto we can see The target URL is also
reachable , it is a basic information for finding additional use for it later

```
nikto -h http://172.16.7.131:3000
```

response

- Nikto v2.6.1

```
-----  
+ Target IP:      172.16.7.131  
+ Target Hostname: 172.16.7.131  
+ Target Port:    3000  
+ Platform:       Unknown  
+ Start Time:     2026-09-09 05:07:59 (GMT-4)
```

5.3 Connectivity validation

```
ping -c 4 172.16.7.133 && curl -i http://172.16.7.133:3000
```

The Response shows a confirmation of the connection to the target host and url

--- 172.16.7.133 ping statistics ---

4 packets transmitted, 4 received, 0% packet loss, time 3037ms

rtt min/avg/max/mdev = 0.692/0.856/1.114/0.176 ms

HTTP/1.1 200 OK

Access-Control-Allow-Origin: *

X-Content-Type-Options: nosniff

X-Frame-Options: SAMEORIGIN

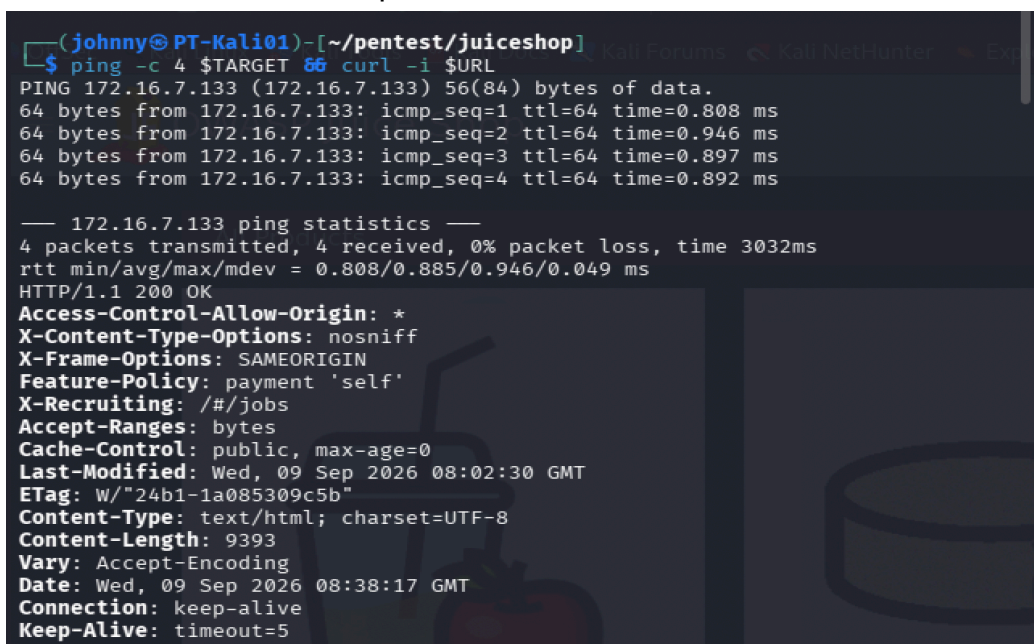
Feature-Policy: payment 'self'

X-Recruiting: /#/jobs

Accept-Ranges: bytes

Cache-Control: public, max-age=0

Last-Modified: Wed, 09 Sep 2026 08:02:30 GMT



```
(johnny@PT-Kali01)-[~/pentest/juiceshop]
$ ping -c 4 $TARGET && curl -i $URL
PING 172.16.7.133 (172.16.7.133) 56(84) bytes of data.
64 bytes from 172.16.7.133: icmp_seq=1 ttl=64 time=0.808 ms
64 bytes from 172.16.7.133: icmp_seq=2 ttl=64 time=0.946 ms
64 bytes from 172.16.7.133: icmp_seq=3 ttl=64 time=0.897 ms
64 bytes from 172.16.7.133: icmp_seq=4 ttl=64 time=0.892 ms

--- 172.16.7.133 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3032ms
rtt min/avg/max/mdev = 0.808/0.885/0.946/0.049 ms
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Accept-Ranges: bytes
Cache-Control: public, max-age=0
Last-Modified: Wed, 09 Sep 2026 08:02:30 GMT
ETag: W/"24b1-1a085309c5b"
Content-Type: text/html; charset=UTF-8
Content-Length: 9393
Vary: Accept-Encoding
Date: Wed, 09 Sep 2026 08:38:17 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```


6.1 Findings and recommendations

This report outlines the findings Secure Ideas collected from the testing, as well as Secure Ideas' recommendations that will assist OWASP Juice Shop in reducing its risks and helping remove the vulnerabilities found.

6.2 overall risk ratings

Each finding is classified as a Critical, High, Medium, or Low risk based on Secure Ideas' professional judgment and experience providing consulting services to organizations of various sizes and industries. In determining risk, Secure Ideas considers each of the following aspects:

- **Potential Threats:** This includes an assessment of potential threat actors and the level of expertise
- **Likelihood of Attack:** Considerations include attacker motivations, complexity of the attack vector, and potentially mitigating security controls
- **Possible Impact:** For each finding, Secure Ideas considers the potential damage to the organization resulting from a successful attack. Each of these factors is assessed individually and in combination to determine the overall risk designation.

These assessments are based on Secure Ideas' professional judgment and experience providing consulting services to enterprises across the country. The following risk level descriptions demonstrate the types of vulnerabilities designated in each category.

Critical

Vulnerabilities found that are being actively exploited in the wild and are known to lead to remote exploitation by external attackers. These security flaws are likely to be targeted and can have a significant impact on the business. These require immediate attention in the form of a workaround or temporary protection. When discovered, Secure Ideas immediately stops all testing and contacts the client for further instructions. Examples of this may include external-facing systems with known remote code execution exploits or remote access interfaces with weak or default credentials.

High

Vulnerabilities found that could lead to exploitation by internal or remote attackers. These security flaws are likely to be targeted and can have a significant impact on the business. These flaws may require immediate attention for temporary protection, but often require more systemic changes in security controls. Some examples include command injection flaws, use of end-of-life software, and default credentials.

Medium

Vulnerabilities or services found that could indirectly contribute to a more major incident; or that are directly exploitable to an extent that is somewhat limited in terms of availability and/or impact. This class of vulnerability is unlikely to lead to a significant compromise on its own, however can pose a substantial danger when combined with others. Some examples include weak transport layer security on a sensitive transaction, insufficient network segmentation, or the use of vulnerable software libraries.

Low

Vulnerabilities or services that, when found alone, are not directly exploitable and present little risk, but may provide information that facilitate the discovery or successful exploitation of other flaws. Examples include disclosure of server software versions and debugging messages.

6.3 Business Impact Summary

Testing identified XSS, SQL Injection, and input validation vulnerabilities in the OWASP Juice Shop application. In a real-world production environment, these weaknesses could allow attackers to execute malicious scripts, compromise user sessions, access or manipulate database information, and interfere with business processes. The resulting impact could include data loss, account compromise, fraud, service disruption, reputational damage, and regulatory or financial consequences. The overall business risk is assessed as High, and remediation should focus on secure database queries, output encoding, server-side validation, and comprehensive retesting.

6.4 Findings Summary

The following table summarizes the findings. Each finding is broken out in detail by risk immediately after the summary table.

Finding	Risk
SQL Injection Flaws	Critical
Weak password configuration policy	Critical
sensitive information disclosure	High
DOM Based XSS	Medium
Username Exposure	Low

7.1 Critical Risk Findings

7.2 SQL Injection Flaws

Industry standards

OWASP Top 10	A05:2025 - Injection
NIST SP 800-53	Improper Sanitization

Summary.

When data enters a web application without being properly sanitized, it may expose the application to several categories of vulnerabilities. One of these categories is the injection of Structured Query Language (SQL) by a third party. This type of attack is commonly referred to as SQL injection.

SQL injection occurs when data is inserted or appended into an application input parameter, and that input is used to dynamically construct a SQL query. When a web application fails to properly sanitize data, which is passed on to dynamically create SQL statements, it is possible for an attacker to alter the construction of back-end SQL statements.

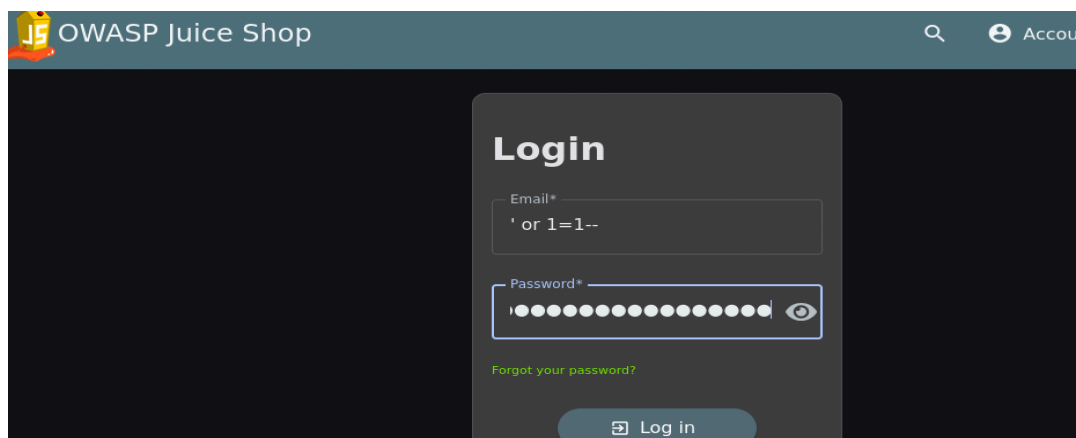
Some of the potential risks include:

- Loss of sensitive or confidential data
- Altered sensitive or confidential data
- Bypass of authentication
- Bypass of authorization
- Access to underlying Operating System
- Further attacks against users of the application (XSS, CSRF) One way to exploit this type of vulnerability is via Blind SQL Injection.

Blind SQL injection is identical to a standard SQL Injection attack, except that when an attacker attempts to exploit an application, rather than getting a useful error message, the attacker instead gets a generic page specified by the developer. This makes exploiting a potential SQL Injection attack more difficult but not impossible. An attacker can still gain access to data by asking a series of True and False questions through SQL statements.

Finding Secure Ideas discovered that the login page of the Juice-shop application is vulnerable to a classic form of SQL Injection as well as Blind SQL Injection. This is due to the use of unsanitized user supplied input. In the first example, using the parameters ' = OR 1=1-- , as the username and any password, Secure Ideas was able to login as the Admin account. Considering Admin was the first user listed in the application, it was therefore used due to the exploit payload.

As shown in the following screenshots, the admin account was the first account listed in the application. Additional accounts could be accessed by using ' or 1=1 and email not like('%admin%');-- and so on.



Below we can see the screenshot of the http post header

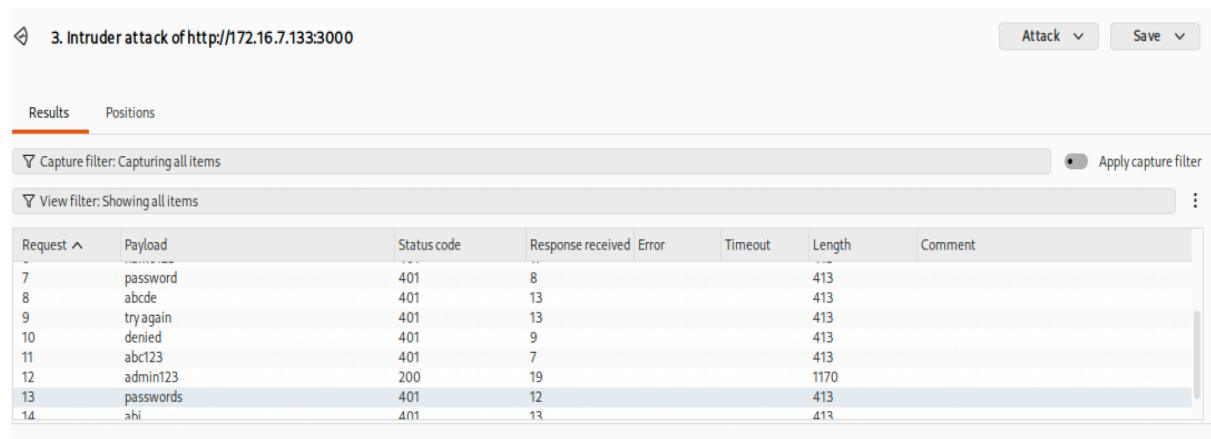
```
'email':" ' or 1=1-- ",  
'password':"commented anyways "
```

7.3 weak password configuration policy

Industry Standards

OWASP Top 10	A07:Authentication Failures
NIST SP 800-63B	Digital Identity Guidelines

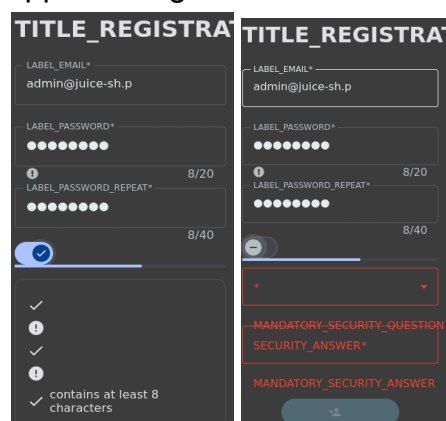
I was able to brute force the admins account
with custom made weak wordlist and got status 200 of success breach



Request ^	Payload	Status code	Response received	Error	Timeout	Length	Comment
7	password	401	8			413	
8	abcde	401	13			413	
9	tryagain	401	13			413	
10	denied	401	9			413	
11	abc123	401	7			413	
12	admin123	200	19			1170	
13	passwords	401	12			413	
14	ahi	401	13			413	

Finding

Finding Secure Ideas found that although the OWASP Juice Shop application does attempt to provide guidelines for secure passwords, it does not adequately enforce the use of complex password configurations. The password complexity criteria are weaker than recommended for an application of this type. The User Registration page shown below displays password advice, however, Secure Ideas found that the application gives this advice as optional, and does not enforce this advice.



TITLE_REGISTRATION

LABEL_EMAIL*
admin@juice-sh.p

LABEL_PASSWORD*
●●●●●●

8/20

LABEL_PASSWORD_REPEAT*
●●●●●●

8/40

☒ Strong

☐ Weak

☐ contains at least 8 characters

TITLE_REGISTRATION

LABEL_EMAIL*
admin@juice-sh.p

LABEL_PASSWORD*
●●●●●●

8/20

LABEL_PASSWORD_REPEAT*
●●●●●●

8/40

☐ Strong

☒ Weak

☐ contains at least 8 characters

☐ MANDATORY_SECURITY_QUESTION

☐ SECURITY_ANSWER*

MANDATORY_SECURITY_ANSWER

Recommendations OWASP Juice Shop should enforce and strengthen the password requirements within the application. While it does perform some complexity checking,

8.1 High Risk Findings

8.2 sensitive information disclosure

Industry Standards

OWASP Top 10	A01:2025: Security Misconfiguration
NIST 800-53	PE-19: Information Leakage

Using Nikto i found out there are accessible folders that hold sensitive information

/ftp/

/.htpasswd

/.bash_history

/.sh_history

```
+ Server: No banner retrieved
+ ERROR: Failed to check for updates: [000000]
+ [000287] /: Retrieved access-control-allow-origin header: \*.
+ [999100] /: Uncommon header(s) 'x-recruiting' found, with contents: /#/job
s.
+ [999997] /robots.txt: Entry '/ftp/' is returned a non-forbidden or redirect
HTTP code (200). See: https://portswigger.net/kb/issues/00600600_robots-tx
t-file
+ [999996] /robots.txt: contains 1 entry which should be manually viewed. Se
e: https://developer.mozilla.org/en-US/docs/Glossary/Robots.txt
+ No CGI Directories found (use '-C all' to force check all possible dirs).
CGI tests skipped.
+ [013587] /: Suggested security header missing: strict-transport-security.
See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transp
ort-Security
+ [013587] /: Suggested security header missing: content-security-policy. Se
e: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
+ [013587] /: Suggested security header missing: permissions-policy. See: ht
tps://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
+ [013587] /: Suggested security header missing: referrer-policy. See: https
://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy
+ [001675] /ftp/: This might be interesting.
+ [001811] /public/: This might be interesting.
+ [002739] /.htpasswd: Contains authorization information.
+ [002743] /.bash_history: A user's home directory may be set to the web roo
t, the shell history was retrieved. This should not be accessible via the we
b.
+ [002756] /.sh_history: A user's home directory may be set to the web root,
the shell history was retrieved. This should not be accessible via the web.
+ ERROR: *** Error limit (20) reached for host, giving up. Last error: openi
ng stream: can't connect (timeout): Transport endpoint is not connected. ***
+ ERROR: *** Consider using mitmproxy to avoid TLS fingerprinting. ***
- STATUS: Completed 7346 requests: currently in plugin 'Nikto Tests'
- STATUS: Running average: Not enough data.
+ Scan terminated: 20 errors and 13 items reported on the remote host
+ End Time: 2026-09-16 06:09:42 (GMT-4) (97 seconds)

+ 1 host(s) tested
```

9.1 Medium Risk Findings

9.2 vulnerability to DOM-based XSS

Industry Standards

OWASP Top 10	A05:2025 Injection
NIST 800-53	Improper Sanitization

The application allows user-controlled HTML to be inserted into the page without proper sanitization. By injecting an iframe containing a JavaScript URL, such as , arbitrary JavaScript executes in the victim's browser. This confirms a DOM-based Cross-Site Scripting vulnerability.

Recommendation:

Properly validate and sanitize all user-supplied HTML before rendering it. Use a trusted HTML sanitizer with an allowlist of permitted tags and attributes, and block dangerous schemes such as javascript:, data:, and vbscript:. Avoid unsafe DOM methods such as innerHTML where possible and use safer alternatives such as textContent. Implement a restrictive Content Security Policy as an additional defense-in-depth measure.

10.1 Low Risk Findings

10.2 Usernames Exposure

Industry Standards

OWASP Top 10	P1 Public Information leakage
NIST 800-53	Security and Privacy Controls

During passive reconnaissance, valid usernames associated with the application were identified from publicly accessible sources.

These usernames may help an attacker construct targeted password-spraying, brute-force, phishing, or credential-stuffing attacks.

disclosure reduces the attacker's effort during account-targeting attacks.

Recommendation:

Avoid publishing internal usernames, administrator names, or predictable account identifiers unnecessarily.

Remove usernames from public documents and HTML comments.

admin@juice-sh.op

basil@juice-sh.op

[bender@juice-sh.op](#)

[stan@juice-sh.op](#)

[uvogin@juice-sh.op](#)

[jim@juice-sh.op](#)

mc.safesearch@juice-sh.op

11.1 Remediation Strategic Guidance

Provide Secure Coding Training for Developers Finding and remediating flaws after the fact is the most expensive way for organizations to handle security vulnerabilities.

It takes a considerable amount of time and effort for developers to consider the discovered issues, review the code, make the appropriate modifications, work through quality assurance testing, and then roll out the changes.

Alternatively, training developers to understand security flaws and avoid vulnerabilities during the development process is much more efficient and effective.

most developer training venues do not adequately teach secure coding.

During this assessment Secure Ideas found evidence that suggests many of OWASP Juice Shop's developers are not properly trained to avoid common mistakes.

OWASP Juice Shop should consider providing secure coding training to all developers on a regular basis.

12.1 Commands Used

Custom wordlist python script
#vim juice_shop_wordlists.py

```
from pathlib import Path
```

```
payloads = [
```

```
    "123",
```

```
    "12345",
```

```
    "11111111",
```

```
    "22222",
```

```
    "123456",
```

```
    "name123",
```

```
    "password",
```

```
    "abcde",
```

```
    "try again",
```

```
    "denied",
```

```
    "abc123",
```

```
    "admin123",
```

```
    "passwords",
```

```
    "abi",
```

```
    "ben",
```

```
]
```

```
Path("juiceshop_wordlist.txt").write_text("\n".join(payloads) + "\n", encoding="utf-8")
```

```
print(f"Wrote {len(payloads)} payloads to juiceshop_wordlist.txt")
```

Run with

```
python3 Juice_Shop_wordlists.py
```